

Implement Client Script & UI Policy (Incident)

Final Project Report

ServiceNow Administrator Micro-Project — SmartBridge

1. Introduction

1.1 Project Overview

The Implement Client Script & UI Policy (Incident) project is a ServiceNow Administrator micro-project that demonstrates how client-side controls — UI Policies and Client Scripts — can be used to enforce data integrity on Incident records. The solution applies conditional field behavior on the Incident form: fields become mandatory or read-only based on Impact, values are auto-populated, and save-time and list-edit-time validation prevent incomplete or ungoverned data changes.

1.2 Purpose

Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. Relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data, impacting reporting accuracy, SLA compliance, and overall service quality. This project enforces conditional field behavior and validation directly at the UI level so that Incident records meet defined business rules before they are submitted.

2. Ideation Phase

2.1 Problem Statement

Incident records are frequently submitted with incomplete or inconsistent data — no assignment group, mismatched urgency relative to impact, and state values changed carelessly from list views without going through the form. This leads to triage delays, SLA misses, and unreliable reporting because there is no enforcement of field behavior at the point of data entry.

2.2 Empathy Map Canvas

Quadrant	Notes
Says	"I just need to log this ticket fast." / "I didn't realize this needed an assignment group." / "Why did my update get blocked?"
Thinks	"I hope I filled everything the manager needs." / "This form has too many fields to remember." / "I don't want to slow down my queue."
Does	Logs incidents quickly under time pressure; sometimes skips fields that don't look mandatory; edits records directly from list views when in a hurry
Feels	Rushed and slightly anxious about missing something; frustrated when a save is rejected without knowing why; relieved when the form guides them automatically

2.3 Brainstorming

Idea	Impact	Effort	Selected?
Agent training only	1	1	No
Business Rule (server-side) validation	3	3	No — no live UI feedback
UI Policy (mandatory/read-only on condition)	5	2	Yes
Client Script onChange (auto-populate Urgency)	4	2	Yes
Client Script onSubmit (save-time validation)	5	2	Yes
Client Script onCellEdit (block list edits)	4	2	Yes
Data Policy only	3	1	No — can't do read-only/list-edit blocking
Workflow/Flow Designer approval	3	4	No — out of scope

3. Requirement Analysis

3.1 Customer Journey Map

Stage	Before	After This Project
Set Impact = High	No guidance on what else is needed	Urgency auto-sets to High + info message shown
Fill remaining fields	May skip Assignment group	Assignment group clearly mandatory; Urgency locked
Submit	Saves with missing Assigned To, discovered later	Blocked immediately with a clear message
List management	Could change State ungoverned	Blocked with alert; must open the record

3.2 Solution Requirement

Functional Requirements

ID	Requirement
FR-01	Assignment group becomes mandatory when Impact = High
FR-02	Urgency becomes read-only when Impact = High
FR-03	Both behaviors reverse automatically when Impact is not High
FR-04	Urgency auto-sets to High (with an info message) when Impact changes to High
FR-05	Save is blocked with an error message if Assigned To is empty on a High Impact incident
FR-06	Inline edits to State from the Incident list are blocked with an alert

Non-Functional Requirements

ID	Requirement
NFR-01	Applies only to the Incident table
NFR-02	Lightweight — no noticeable form load/save performance impact
NFR-03	Fully configurable/maintainable by an admin, no schema changes
NFR-04	Works on out-of-the-box Incident fields (impact, urgency, assignment_group, assigned_to, state)

3.3 Data Flow Diagram

Agent sets Impact on the Incident Form -> UI Policy 'High Impact Control' evaluates the condition -> UI Policy Actions set Assignment group to Mandatory and Urgency to Read-only. In parallel, the onChange Client Script auto-sets Urgency and shows an info message. On Submit, the onSubmit Client Script checks Assigned To and either blocks the save with an error or allows the record to save. Separately, on the Incident list view, the onCellEdit Client Script intercepts any inline edit to State, shows an alert, and rejects the edit; State changes made from the form save normally.

3.4 Technology Stack

Layer	Technology
Platform	ServiceNow (Now Platform)
Application	ITSM — Incident Management
Client-side logic	Client Scripts (JavaScript: onChange, onSubmit, onCellEdit) — GlideForm API
Declarative configuration	UI Policies & UI Policy Actions
Development environment	ServiceNow Personal Developer Instance (PDI)
Version control	GitHub (public repository, phase-wise structure)

4. Project Design

4.1 Problem Solution Fit

The problem — inconsistent, incomplete Incident data caused by manual data entry — is directly addressed by moving enforcement to the client side of the form and list, where the agent is already working. A UI Policy handles

the declarative, condition-based rules (mandatory/read-only), while targeted Client Scripts cover the logic a UI Policy cannot express: auto-population, custom save-time validation, and blocking risky inline list edits. Together they close every gap identified in the problem statement without adding a separate approval process or extra steps for the agent.

4.2 Proposed Solution

- UI Policy 'High Impact Control' — triggers when Impact = High; reverses automatically when it is not.
- UI Policy Action — Assignment group becomes Mandatory.
- UI Policy Action — Urgency becomes Read-only.
- Client Script (onChange) — auto-sets Urgency to High with an info message.
- Client Script (onSubmit) — blocks save if Assigned To is empty on a High Impact incident.
- Client Script (onCellEdit) — blocks direct State edits from the Incident list view.

4.3 Solution Architecture

Incident Form Loads -> User sets/changes Impact -> onChange Client Script auto-sets Urgency (with message) while the UI Policy 'High Impact Control' evaluates its condition in parallel -> UI Policy Actions set Assignment group to Mandatory and Urgency to Read-only (both reverse automatically if Impact is no longer High) -> User clicks Submit -> onSubmit Client Script checks Assigned To and blocks the save with an error if it is empty, otherwise the record saves -> Separately, the Incident list view is protected by the onCellEdit Client Script, which blocks direct State edits and requires the agent to open the record instead.

5. Project Planning & Scheduling

5.1 Project Planning

Task	Description
Task 1	Create UI Policy 'High Impact Control' (Table = Incident, Condition = Impact is High, Reverse if false = true)
Task 2	Create UI Policy Actions — Assignment group (Mandatory), Urgency (Read-only)
Task 3	Create onChange Client Script — auto-set Urgency
Task 4	Create onSubmit Client Script — block save if Assigned To missing
Task 5	Create onCellEdit Client Script — block State list-edit
Task 6	Test full configuration across all scenarios

Build order follows a strict dependency chain: the UI Policy is created before its actions, actions are configured before the Client Scripts, the onChange script is built and tested before the onSubmit script, and the onCellEdit script — which operates on the list view rather than the form — is built and tested last, in isolation. Full end-to-end testing only happens once every component is active.

6. Functional and Performance Testing

6.1 Performance Testing

Test Area	Method	Result
Mandatory field enforcement	Set Impact = High, leave Assignment group / Assigned To empty, attempt save	Pass — save blocked as expected
Read-only enforcement	Set Impact = High, attempt to edit Urgency	Pass — field locked
Auto-population	Change Impact to High	Pass — Urgency auto-set with info message
Reverse-if-false behavior	Change Impact from High to Medium on an existing record	Pass — mandatory/read-only reverted
List-edit blocking	Double-click State in the Incident list and attempt an edit	Pass — alert shown, edit rejected

Form-based update	Change State from the form and save	Pass — saved successfully
Form load performance	Open Incident form with policy/scripts active	Pass — no noticeable delay

7. Results

7.1 Output Screenshots

[Insert screenshots here: the High Impact Control UI Policy record, its two UI Policy Actions, the three Client Script records, a High Impact incident with Urgency auto-set and Assignment group marked mandatory, the onSubmit error box blocking a save, and the onCellEdit alert blocking a list edit.]

8. Advantages & Disadvantages

Advantages

- Enforces data quality at the point of entry rather than after the fact.
- No custom tables or schema changes required — works entirely on out-of-the-box fields.
- Declarative UI Policy is easy for any admin to read, maintain, and extend.
- Reverse if false keeps the form correctly relaxed for non-High-impact incidents.
- Closes the list-view loophole that would otherwise bypass form-level governance.

Disadvantages

- Purely client-side enforcement can be bypassed via direct API/Import Set updates that don't go through the browser form.
- Hardcoded field/value checks (e.g., Impact = '1') require updates if choice list values change.
- onCellEdit blocking may frustrate agents used to quick inline list edits until they learn the new behavior.

9. Conclusion

The Implement Client Script & UI Policy (Incident) micro-project effectively demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. By using onChange, onSubmit, and onCellEdit scripts alongside UI Policy actions, the project ensures fields like Assigned To and Urgency respond appropriately to changes in Impact, maintaining clean and consistent data. This solution enforces mandatory fields and controls user interactions while remaining lightweight, efficient, and easy to implement — a practical example of improving form usability and data integrity in ServiceNow.

10. Future Scope

- Extend the same pattern to other task tables (Problem, Change Request) for consistent data governance.
- Add server-side Business Rules or ACLs to close the client-side-only enforcement gap for API/Import Set updates.
- Move hardcoded choice values to system properties for easier maintenance.
- Add automated test coverage using ATF (Automated Test Framework) for regression-safe changes.
- Extend UI Policy conditions to cover additional fields (e.g., Category, Short description length) as new data-quality issues are identified.